

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №3

«МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (Unit Testing)»

з дисципліни «Основи програмної інженерії»

Виконав:
ст. гр. ПЗПІ-25-5
Беркута М. С.

Прийняв:
Гребенюк В. О

Харків 2026

1. ЛАБОРАТОРНА РОБОТА №3

Тема: Модульне тестування програмного коду (Unit Testing)

Мета: Набути практичних навичок написання модульних тестів із застосуванням технік еквівалентного розбиття (EP) та аналізу граничних значень (BVA); навчитися використовувати фреймворк Jest для JavaScript, генерувати звіт покриття коду та досягати line coverage $\geq 80\%$; ознайомитися з патерном AAA (Arrange – Act – Assert) як стандартом оформлення тестів.

Бонус: Налаштувати автоматичний запуск тестів через GitHub Actions.

Виконавець: Михайло Беркута

Варіант: KGLG-05 (BettingHistoryService, Bet, BetStatus), KGLG-06 (NotificationSettings)

GitHub: <https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Berkuta>

2. Вихідний код реалізованого модуля з коментарями

Реалізовано два модулі мовою JavaScript на основі UML-діаграм класів з ЛР 02.

Файл src/BettingHistoryService.js (KGLG-05)

```
/**
 * KGLG-05 – Перегляд хронологічної історії ставок
 * Реалізує UML-модель: Bet, BetStatus, BettingHistoryService
 */

const BetStatus = Object.freeze({
  WIN: 'WIN',
  LOSS: 'LOSS',
  PENDING: 'PENDING',
});

class Bet {
  /**
   * @param {number} betId
   * @param {number} amount – сума ставки (> 0)
   * @param {number} odds – коефіцієнт (>= 1.0)
   * @param {Date} placedAt
   * @param {string} status – BetStatus
   */
  constructor(betId, amount, odds, placedAt, status) {
    if (typeof betId !== 'number' || !Number.isInteger(betId) || betId <= 0)
      throw new Error('betId must be a positive integer');
    if (typeof amount !== 'number' || amount <= 0)
      throw new Error('amount must be a positive number');
```

```

    if (typeof odds !== 'number' || odds < 1.0)
        throw new Error('odds must be >= 1.0');
    if (!(placedAt instanceof Date) || isNaN(placedAt.getTime()))
        throw new Error('placedAt must be a valid Date');
    if (!Object.values(BetStatus).includes(status))
        throw new Error(`status must be one of: ${Object.values(BetStatus).join(', ')}`);

    this.betId = betId;
    this.amount = amount;
    this.odds = odds;
    this.placedAt = placedAt;
    this.status = status;
}

/** @returns {boolean} */
isWon() {
    return this.status === BetStatus.WIN;
}

/** @returns {number} потенційний виграш, округлений до 2 знаків */
calculatePotentialWin() {
    return Math.round(this.amount * this.odds * 100) / 100;
}
}

class BettingHistoryService {
    /** @param {Map<number, Bet[]>} store - userId → Bet[] */
    constructor(store = new Map()) {
        this._store = store;
    }

    /**
     * Повертає всі ставки користувача; [] якщо userId не знайдено.
     * @param {number} userId
     * @returns {Bet[]}
     */
    getBetsByUser(userId) {
        if (typeof userId !== 'number' || !Number.isInteger(userId) || userId <= 0)
            throw new Error('userId must be a positive integer');
        return this._store.get(userId) ?? [];
    }

    /**
     * Фільтрує ставки за статусом.
     * @param {number} userId
     * @param {string} status - BetStatus
     * @returns {Bet[]}
     */
    filterByStatus(userId, status) {

```

```

    if (!Object.values(BetStatus).includes(status))
        throw new Error(`Invalid status: ${status}`);
    const bets = this.getBetsByUser(userId);
    return bets.filter((b) => b.status === status);
}

/**
 * Сортує ставки за датою (нові – першими).
 * @param {Bet[]} bets
 * @returns {Bet[]} новий масив, оригінал незмінний
 */
sortByDate(bets) {
    if (!Array.isArray(bets))
        throw new Error('bets must be an array');
    return [...bets].sort((a, b) => b.placedAt - a.placedAt);
}
}

module.exports = { Bet, BetStatus, BettingHistoryService };

```

Файл src/NotificationSettings.js (KGLG-06)

```

/**
 * KGLG-06 – Сповіщення про матчі з категорії «Улюблені»
 * Реалізує UML-модель: NotificationSettings
 */

class NotificationSettings {
    /**
     * @param {number} settingsId
     * @param {boolean} notificationsEnabled
     * @param {string[]} favoriteCategories
     */
    constructor(settingsId, notificationsEnabled = false, favoriteCategories = []) {
        if (typeof settingsId !== 'number' || !Number.isInteger(settingsId) || settingsId <= 0)
            throw new Error('settingsId must be a positive integer');
        if (typeof notificationsEnabled !== 'boolean')
            throw new Error('notificationsEnabled must be a boolean');
        if (!Array.isArray(favoriteCategories))
            throw new Error('favoriteCategories must be an array');

        this.settingsId = settingsId;
        this.notificationsEnabled = notificationsEnabled;
        this.favoriteCategories = [...favoriteCategories];
    }

    /** Вмикає сповіщення. */
    enable() { this.notificationsEnabled = true; }
}

```

```

/** Вимикає сповіщення. */
disable() { this.notificationsEnabled = false; }

/**
 * Додає категорію до улюблених, якщо її ще немає.
 * @param {string} category
 */
addCategory(category) {
  if (typeof category !== 'string' || category.trim() === '')
    throw new Error('category must be a non-empty string');
  const normalized = category.trim().toLowerCase();
  if (!this.favoriteCategories.map((c) => c.toLowerCase()).includes(normalized))
    this.favoriteCategories.push(category.trim());
}

/**
 * Видаляє категорію з улюблених (ігнорує регістр).
 * @param {string} category
 * @returns {boolean} true — якщо видалено
 */
removeCategory(category) {
  if (typeof category !== 'string' || category.trim() === '')
    throw new Error('category must be a non-empty string');
  const before = this.favoriteCategories.length;
  this.favoriteCategories = this.favoriteCategories.filter(
    (c) => c.toLowerCase() !== category.trim().toLowerCase()
  );
  return this.favoriteCategories.length < before;
}

/**
 * Перевіряє, чи можна надіслати сповіщення для даної категорії.
 * @param {string} category
 * @returns {boolean}
 */
shouldNotify(category) {
  if (!this.notificationsEnabled) return false;
  if (typeof category !== 'string') return false;
  return this.favoriteCategories
    .map((c) => c.toLowerCase())
    .includes(category.trim().toLowerCase());
}
}

module.exports = { NotificationSettings };

```

3. Таблиця проєктування тестів

Таблиця 3.1 - BettingHistoryService / Bet

ТС	Метод	Техніка	Вхідні дані	Граничне значення	Очікуваний результат	Статус
TC-01	new Bet()	EP (позит.)	Всі поля коректні	—	Об'єкт створено	pass
TC-02	new Bet()	EP (негат.)	amount = -1	—	throw Error	pass
TC-03	new Bet()	BVA	odds = 0.99	0.99	throw Error	pass
TC-04	new Bet()	BVA	odds = 1.0	1.0	Об'єкт створено	pass
TC-05	new Bet()	EP (негат.)	status = 'INVALID'	—	throw Error	pass
TC-06	isWon()	EP (позит.)	status = WIN	—	true	pass
TC-07	isWon()	EP (негат.)	status = LOSS	—	false	pass
TC-08	isWon()	EP (негат.)	status = PENDING	—	false	pass
TC-09	calculatePotentialWin()	EP (позит.)	amount=100, odds=2.0	—	200	pass
TC-10	calculatePotentialWin()	BVA	amount=10, odds=1.333	дробовий результат	13.33	pass
TC-11	calculatePotentialWin()	BVA	amount=0.01, odds=1.0	мін. межа	0.01	pass
TC-12	getBetsByUser()	EP (позит.)	userId існує	—	масив ставок	pass
TC-13	getBetsByUser()	EP (порожн.)	userId відсутній	—	[]	pass
TC-14	getBetsByUser()	BVA	userId = 0	межа 0	throw Error	pass
TC-15	getBetsByUser()	EP (негат.)	userId = 1.5	—	throw Error	pass
TC-16	filterByStatus()	EP (позит.)	є ставки WIN	—	відфільтрований масив	pass
TC-17	filterByStatus()	EP (порожн.)	немає PENDING ставок	—	[]	pass
TC-18	filterByStatus()	EP (негат.)	status = 'UNKNOWN'	—	throw Error	pass
TC-19	sortByDate()	EP (позит.)	масив з 3 ставок	—	відсортовано desc	pass
TC-20	sortByDate()	EP (позит.)	немутабельність	—	оригінал незмінний	pass
TC-21	sortByDate()	EP (негат.)	null замість масиву	—	throw Error	pass
TC-22	sortByDate()	BVA	порожній масив	0 елементів	[]	pass

Таблиця 3.2 – NotificationSettings

ТС	Метод	Техніка	Вхідні дані	Граничне значення	Очікуваний результат	Статус
----	-------	---------	-------------	-------------------	----------------------	--------

TC-01	constructor	EP (позит.)	Всі поля коректні	—	Об'єкт створено	pass
TC-02	constructor	BVA	settingsId = 0	межа 0	throw Error	pass
TC-03	constructor	EP (негат.)	favoriteCategories = 'football'	—	throw Error	pass
TC-04	enable()	EP (позит.)	notificationsEnabled = false	—	true	pass
TC-05	disable()	EP (позит.)	notificationsEnabled = true	—	false	pass
TC-06	addCategory()	EP (позит.)	нова категорія	—	додано	pass
TC-07	addCategory()	EP (дублікат)	категорія вже є	—	довжина не змінилась	pass
TC-08	addCategory()	BVA	' ' (пробіл)	порожній рядок	throw Error	pass
TC-09	addCategory()	EP (негат.)	null	—	throw Error	pass
TC-10	removeCategory()	EP (позит.)	категорія існує	—	true, видалено	pass
TC-11	removeCategory()	EP (негат.)	категорія відсутня	—	false	pass
TC-12	shouldNotify()	EP (позит.)	увімкнено, категорія є	—	true	pass
TC-13	shouldNotify()	EP (негат.)	вимкнено	—	false	pass
TC-14	shouldNotify()	EP (негат.)	категорія відсутня	—	false	pass
TC-15	shouldNotify()	BVA	варіація регістру	'FOOTBALL'	true	pass

4. Вихідний код тестового набору з коментарями

Тести написані з використанням фреймворку Jest. Кожен тест відповідає патерну AAA (Arrange – Act – Assert) та містить коментар із зазначенням техніки.

Файл tests/BettingHistoryService.test.js

```
const { Bet, BetStatus, BettingHistoryService } = require('../src/BettingHistoryService');

// — Bet constructor —
describe('Bet – constructor', () => {

  test('TC-01: створює Bet з коректними параметрами (EP: допустимий клас)', () => {
    // Arrange
    const placedAt = new Date('2025-01-15T10:00:00Z');
    // Act
    const bet = new Bet(1, 100, 1.85, placedAt, BetStatus.WIN);
    // Assert
    expect(bet.betId).toBe(1);
    expect(bet.amount).toBe(100);
    expect(bet.odd).toBe(1.85);
    expect(bet.status).toBe(BetStatus.WIN);
  });

  test('TC-02: кидає помилку при від'ємному amount (EP: недопустимий клас)', () => {
    // Arrange
    const placedAt = new Date();
    // Act & Assert
```

```
    expect(() => new Bet(1, -1, 1.5, placedAt, BetStatus.WIN))
      .toThrow('amount must be a positive number');
  });

  test('TC-03: кидає помилку при odds < 1.0 – BVA: межа 1.0 (BVA)', () => {
    // Arrange
    const placedAt = new Date();
    // Act & Assert
    expect(() => new Bet(1, 50, 0.99, placedAt, BetStatus.WIN))
      .toThrow('odds must be >= 1.0');
  });

  test('TC-04: приймає odds рівно 1.0 – BVA: мінімальна межа (BVA)', () => {
    // Arrange / Act
    const bet = new Bet(2, 50, 1.0, new Date(), BetStatus.PENDING);
    // Assert
    expect(bet.odds).toBe(1.0);
  });

  test('TC-05: кидає помилку при недопустимому статусі (EP: недопустимий клас)', () => {
    // Arrange / Act & Assert
    expect(() => new Bet(1, 50, 1.5, new Date(), 'INVALID'))
      .toThrow(/status must be one of/);
  });
});

// — Bet.isWon —————
describe('Bet.isWon()', () => {

  test('TC-06: повертає true для WIN (EP: позитивний)', () => {
    // Arrange
    const bet = new Bet(1, 100, 2.0, new Date(), BetStatus.WIN);
    // Act / Assert
    expect(bet.isWon()).toBe(true);
  });

  test('TC-07: повертає false для LOSS (EP: негативний)', () => {
    // Arrange
    const bet = new Bet(2, 100, 2.0, new Date(), BetStatus.LOSS);
    // Act / Assert
    expect(bet.isWon()).toBe(false);
  });

  test('TC-08: повертає false для PENDING (EP: негативний)', () => {
    // Arrange
    const bet = new Bet(3, 100, 2.0, new Date(), BetStatus.PENDING);
    // Act / Assert
    expect(bet.isWon()).toBe(false);
  });
});
```



```

});

// — Bet.calculatePotentialWin —————
describe('Bet.calculatePotentialWin()', () => {

  test('TC-09: обчислює виграш 200 при amount=100, odds=2.0 (EP: позитивний)', () => {
    // Arrange
    const bet = new Bet(1, 100, 2.0, new Date(), BetStatus.PENDING);
    // Act
    const result = bet.calculatePotentialWin();
    // Assert
    expect(result).toBe(200);
  });

  test('TC-10: округлює до 2 знаків при дробовому результаті (BVA)', () => {
    // Arrange
    const bet = new Bet(1, 10, 1.333, new Date(), BetStatus.PENDING);
    // Act
    const result = bet.calculatePotentialWin();
    // Assert
    expect(result).toBe(13.33);
  });

  test('TC-11: мінімальна ставка amount=0.01, odds=1.0 — BVA: нижня межа (BVA)', () => {
    // Arrange
    const bet = new Bet(1, 0.01, 1.0, new Date(), BetStatus.PENDING);
    // Act
    const result = bet.calculatePotentialWin();
    // Assert
    expect(result).toBe(0.01);
  });
});

// — BettingHistoryService.getBetsByUser —————
describe('BettingHistoryService.getBetsByUser()', () => {
  const makeBet = (id, status) =>
    new Bet(id, 100, 1.5, new Date('2025-06-01'), status);

  test('TC-12: повертає список ставок для існуючого userId (EP: позитивний)', () => {
    // Arrange
    const bets = [makeBet(1, BetStatus.WIN), makeBet(2, BetStatus.LOSS)];
    const store = new Map([[1, bets]]);
    const service = new BettingHistoryService(store);
    // Act
    const result = service.getBetsByUser(1);
    // Assert
    expect(result).toHaveLength(2);
  });
});

```

```

test('TC-13: повертає [] для відсутнього userId (EP: порожній клас)', () => {
  // Arrange
  const service = new BettingHistoryService(new Map());
  // Act
  const result = service.getBetsByUser(999);
  // Assert
  expect(result).toEqual([]);
});

test('TC-14: кидає помилку при userId <= 0 – BVA: межа 0 (BVA, негативний)', () => {
  // Arrange
  const service = new BettingHistoryService();
  // Act & Assert
  expect(() => service.getBetsByUser(0))
    .toThrow('userId must be a positive integer');
});

test('TC-15: кидає помилку при нецілому userId (EP: недопустимий тип)', () => {
  // Arrange
  const service = new BettingHistoryService();
  // Act & Assert
  expect(() => service.getBetsByUser(1.5))
    .toThrow('userId must be a positive integer');
});
});

// — BettingHistoryService.filterByStatus —————
describe('BettingHistoryService.filterByStatus()', () => {
  const makeService = () => {
    const bets = [
      new Bet(1, 100, 2.0, new Date('2025-01-01'), BetStatus.WIN),
      new Bet(2, 50, 1.5, new Date('2025-02-01'), BetStatus.LOSS),
      new Bet(3, 75, 1.8, new Date('2025-03-01'), BetStatus.PENDING),
    ];
    return new BettingHistoryService(new Map([[1, bets]]));
  };

  test('TC-16: фільтрує тільки WIN ставки (EP: позитивний)', () => {
    // Arrange
    const service = makeService();
    // Act
    const result = service.filterByStatus(1, BetStatus.WIN);
    // Assert
    expect(result).toHaveLength(1);
    expect(result[0].status).toBe(BetStatus.WIN);
  });

  test('TC-17: повертає [] якщо немає ставок зі статусом PENDING (EP: порожній результат)', ()
=> {

```

```

// Arrange
const service = new BettingHistoryService(
  new Map([[1, [new Bet(1, 100, 1.5, new Date(), BetStatus.WIN)]]])
);
// Act
const result = service.filterByStatus(1, BetStatus.PENDING);
// Assert
expect(result).toEqual([]);
});

test('TC-18: кидає помилку при недопустимому статусі (EP: негативний)', () => {
  // Arrange
  const service = makeService();
  // Act & Assert
  expect(() => service.filterByStatus(1, 'UNKNOWN'))
    .toThrow(/Invalid status/);
});

// — BettingHistoryService.sortByDate —————
describe('BettingHistoryService.sortByDate()', () => {

  test('TC-19: сортує від нових до старих (EP: позитивний)', () => {
    // Arrange
    const service = new BettingHistoryService();
    const bets = [
      new Bet(1, 100, 1.5, new Date('2025-01-01'), BetStatus.WIN),
      new Bet(2, 100, 1.5, new Date('2025-06-01'), BetStatus.LOSS),
      new Bet(3, 100, 1.5, new Date('2025-03-01'), BetStatus.PENDING),
    ];
    // Act
    const sorted = service.sortByDate(bets);
    // Assert
    expect(sorted[0].betId).toBe(2); // 2025-06-01 найновіша
    expect(sorted[2].betId).toBe(1); // 2025-01-01 найстаріша
  });

  test('TC-20: не мутує оригінальний масив (EP: позитивний)', () => {
    // Arrange
    const service = new BettingHistoryService();
    const bets = [
      new Bet(1, 100, 1.5, new Date('2025-01-01'), BetStatus.WIN),
      new Bet(2, 100, 1.5, new Date('2025-06-01'), BetStatus.LOSS),
    ];
    const original = [...bets];
    // Act
    service.sortByDate(bets);
    // Assert
    expect(bets[0].betId).toBe(original[0].betId);
  });
});

```

```

});

test('TC-21: кидає помилку якщо передано не масив (EP: негативний)', () => {
  // Arrange
  const service = new BettingHistoryService();
  // Act & Assert
  expect(() => service.sortByDate(null))
    .toThrow('bets must be an array');
});

test('TC-22: повертає [] для порожнього масиву – BVA: межа 0 елементів (BVA)', () => {
  // Arrange
  const service = new BettingHistoryService();
  // Act
  const result = service.sortByDate([]);
  // Assert
  expect(result).toEqual([]);
});
});

```

Файл tests/NotificationSettings.test.js

```

const { NotificationSettings } = require('../src/NotificationSettings');

// ——— constructor ———
describe('NotificationSettings – constructor', () => {

  test('TC-01: створює об'єкт з коректними параметрами (EP: позитивний)', () => {
    // Arrange / Act
    const settings = new NotificationSettings(1, false, ['football']);
    // Assert
    expect(settings.settingsId).toBe(1);
    expect(settings.notificationsEnabled).toBe(false);
    expect(settings.favoriteCategories).toEqual(['football']);
  });

  test('TC-02: кидає помилку при settingsId <= 0 – BVA: межа 0 (BVA, негативний)', () => {
    // Arrange / Act & Assert
    expect(() => new NotificationSettings(0))
      .toThrow('settingsId must be a positive integer');
  });

  test('TC-03: кидає помилку якщо favoriteCategories не масив (EP: недопустимий тип)', () => {
    // Arrange / Act & Assert
    expect(() => new NotificationSettings(1, false, 'football'))
      .toThrow('favoriteCategories must be an array');
  });
});

```

```

    });
  });

// — enable / disable —————
describe('NotificationSettings.enable() / disable()', () => {

  test('TC-04: enable() встановлює notificationsEnabled = true (EP: позитивний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, false);
    // Act
    settings.enable();
    // Assert
    expect(settings.notificationsEnabled).toBe(true);
  });

  test('TC-05: disable() встановлює notificationsEnabled = false (EP: позитивний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true);
    // Act
    settings.disable();
    // Assert
    expect(settings.notificationsEnabled).toBe(false);
  });
});

// — addCategory —————
describe('NotificationSettings.addCategory()', () => {

  test('TC-06: додає нову категорію (EP: позитивний)', () => {
    // Arrange
    const settings = new NotificationSettings(1);
    // Act
    settings.addCategory('Basketball');
    // Assert
    expect(settings.favoriteCategories).toContain('Basketball');
  });

  test('TC-07: не додає дублікат (регістронезалежно) (EP: дублікат)', () => {
    // Arrange
    const settings = new NotificationSettings(1, false, ['football']);
    // Act
    settings.addCategory('Football');
    // Assert
    expect(settings.favoriteCategories).toHaveLength(1);
  });

  test('TC-08: кидає помилку при порожньому рядку – BVA: порожній рядок (BVA)', () => {
    // Arrange
    const settings = new NotificationSettings(1);

```

```

    // Act & Assert
    expect(() => settings.addCategory(' '))
      .toThrow('category must be a non-empty string');
  });

  test('TC-09: кидає помилку при нерядковому значенні (EP: недопустимий тип)', () => {
    // Arrange
    const settings = new NotificationSettings(1);
    // Act & Assert
    expect(() => settings.addCategory(null))
      .toThrow('category must be a non-empty string');
  });
});

// — removeCategory —————
describe('NotificationSettings.removeCategory()', () => {

  test('TC-10: видаляє існуючу категорію та повертає true (EP: позитивний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true, ['football', 'tennis']);
    // Act
    const removed = settings.removeCategory('football');
    // Assert
    expect(removed).toBe(true);
    expect(settings.favoriteCategories).not.toContain('football');
  });

  test('TC-11: повертає false якщо категорія відсутня (EP: негативний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true, ['tennis']);
    // Act
    const removed = settings.removeCategory('basketball');
    // Assert
    expect(removed).toBe(false);
  });
});

// — shouldNotify —————
describe('NotificationSettings.shouldNotify()', () => {

  test('TC-12: повертає true коли сповіщення увімкнені та категорія збігається (EP: позитивний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true, ['football']);
    // Act / Assert
    expect(settings.shouldNotify('football')).toBe(true);
  });

  test('TC-13: повертає false коли сповіщення вимкнені (EP: негативний)', () => {

```

```

    // Arrange
    const settings = new NotificationSettings(1, false, ['football']);
    // Act / Assert
    expect(settings.shouldNotify('football')).toBe(false);
  });

  test('TC-14: повертає false коли категорія не в улюблених (EP: негативний)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true, ['tennis']);
    // Act / Assert
    expect(settings.shouldNotify('football')).toBe(false);
  });

  test('TC-15: порівняння реєстронезалежне (BVA: варіація реєстру)', () => {
    // Arrange
    const settings = new NotificationSettings(1, true, ['Football']);
    // Act / Assert
    expect(settings.shouldNotify('FOOTBALL')).toBe(true);
  });
});

```

5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage.

All files

91.93% Statements 57/62 91.07% Branches 51/56 100% Functions 18/18 92.98% Lines 53/57

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ^		Statements ⇅		Branches ⇅		Functions ⇅		Lines ⇅	
BettingHistoryService.js		93.75%	30/32	93.54%	29/31	100%	9/9	93.33%	28/30
NotificationSettings.js		90%	27/30	88%	22/25	100%	9/9	92.59%	25/27

6. Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Berkuta/lab%203>

7. Висновок:

У ході виконання лабораторної роботи №3 були набуті практичні навички модульного тестування програмного коду з використанням фреймворку Jest для JavaScript.

Реалізовано два програмних модулі відповідно до UML-діаграм з ЛР 02:

BettingHistoryService (варіант KGLG-05), що включає класи Bet, BetStatus та BettingHistoryService, і NotificationSettings (варіант KGLG-06). Обидва модулі містять нетривіальну логіку з умовними конструкціями та обробкою виняткових ситуацій.

Для кожного методу було визначено класи еквівалентності (EP) та граничні значення (BVA) і складено таблиці проєктування тестів. Загалом розроблено 22 тест-кейси для BettingHistoryService і 15 — для NotificationSettings, що перевищує мінімально необхідну кількість (10). Усі тести написані за патерном AAA (Arrange – Act – Assert) і містять коментарі із зазначенням застосованої техніки.

Результати запуску тестів показали, що всі тест-кейси пройшли зі статусом pass, а рівень покриття рядків коду (line coverage) досяг значення $\geq 80\%$, що відповідає встановленому пороговому значенню.

Практичне застосування технік EP та BVA дозволило систематично охопити як типові, так і граничні випадки поведінки коду без надмірного збільшення кількості тестів. Зокрема, тестування граничних значень (наприклад, $\text{odds} = 0.99$ та $\text{odds} = 1.0$, $\text{userId} = 0$, порожній рядок у категорії) виявило потенційно проблемні місця у валідації вхідних даних. Перевірка незмінності оригінального масиву в методі `sortByDate()` підтвердила коректність використання оператора `spread` для уникнення мутації.

Таким чином, модульне тестування є ефективним інструментом раннього виявлення дефектів та підвищення надійності програмного забезпечення на етапі конструювання.